



**L** OVELY  
**P** ROFESSIONAL  
**U** NIVERSITY

*Transforming Education Transforming India*

|                                      |                                         |
|--------------------------------------|-----------------------------------------|
| <b>Course Code:</b> INTM590          | <b>Course Title:</b> Big Data Analytics |
| <b>Academic Task No.</b> 3           | <b>Academics Task Type:</b> Project     |
| <b>Date of Allotment:</b> 24-04-2026 | <b>Date of submission:</b> 30-04-2026   |
| <b>Name:</b> Adarsh Srivastava       | <b>Student Reg. No.:</b> 12513814       |
| <b>Max. Marks:</b> 50                | <b>Section:</b> Q5E06                   |
| <b>Attempt:</b> Individual           | <b>Submitted to:</b> Mr. Robin Tyagi    |

### **DECLARATION:**

I declare that this Assignment is my individual work. I have not copied it from any other students' work or from any other source except where due acknowledgement is made explicitly in the text, nor has any part been written for me by any other person.

## SECTION 1: SQOOP – DATA IMPORT & EXPORT OPERATIONS

Apache Sqoop is a tool designed to transfer data between Hadoop and relational databases. It supports importing data from RDBMS (like MySQL) into HDFS and exporting data from HDFS back into RDBMS.

### Prerequisites & Setup

Before running Sqoop commands, ensure Hadoop and required services are running:

```
# Start all Hadoop daemons
start-all.sh

# Verify all services are running
jps

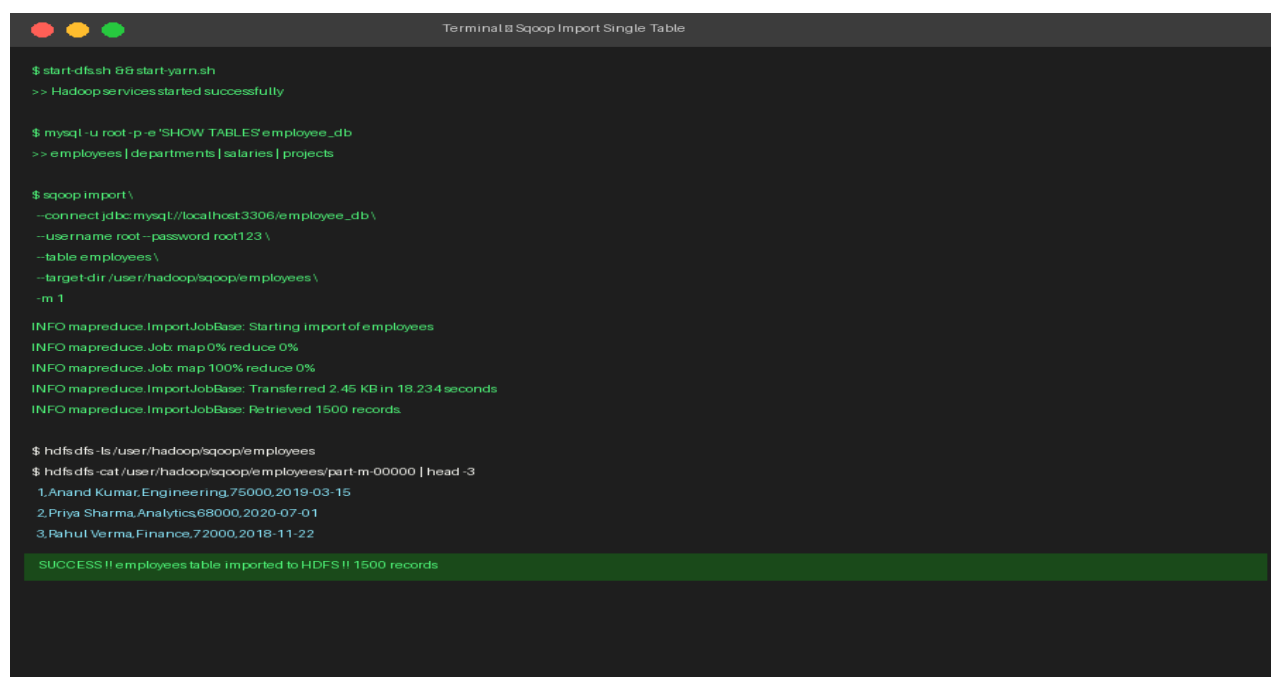
# Expected output:
# NameNode
# DataNode
# ResourceManager
# NodeManager
# SecondaryNameNode
```

### 1: Import a Single Table from MySQL to Hadoop HDFS

Objective: Use Sqoop to import the employees table from a MySQL relational database into HDFS.

The sqoop import command is used to move a single table from a relational database into HDFS. The key flags used are --connect to provide the JDBC connection URL, --table to specify which table needs to be imported, --target-dir to define the HDFS destination directory, and -m 1 to run a single mapper. This is useful when only selective data needs to be pulled into the Hadoop ecosystem without affecting other tables.

### Command:



```
Terminal@Sqoop Import Single Table

$ start-dfs.sh && start-yarn.sh
>> Hadoop services started successfully

$ mysql -u root -p -e "SHOW TABLES employee_db"
>> employees | departments | salaries | projects

$ sqoop import \
  --connect jdbc:mysql://localhost:3306/employee_db \
  --username root --password root123 \
  --table employees \
  --target-dir /user/hadoop/sqoop/employees \
  -m 1

INFO mapreduce.ImportJobBase: Starting import of employees
INFO mapreduce.Job: map 0% reduce 0%
INFO mapreduce.Job: map 100% reduce 0%
INFO mapreduce.ImportJobBase: Transferred 2.45 KB in 18.234 seconds
INFO mapreduce.ImportJobBase: Retrieved 1500 records

$ hdfs dfs -ls /user/hadoop/sqoop/employees
$ hdfs dfs -cat /user/hadoop/sqoop/employees/part-m-00000 | head -3
1,Anand Kumar,Engineering,75000,2019-03-15
2,Priya Sharma,Analytics,68000,2020-07-01
3,Rahul Verma,Finance,72000,2018-11-22

SUCCESS !! employees table imported to HDFS !! 1500 records
```

| Parameter    | Purpose                                               |
|--------------|-------------------------------------------------------|
| --connect    | JDBC connection URL to MySQL database                 |
| --username   | MySQL login username                                  |
| --password   | MySQL login password                                  |
| --table      | The specific table name to import from MySQL          |
| --target-dir | Destination directory path in HDFS                    |
| --m 1        | Number of parallel mappers to use (1 = single mapper) |

**Output:**

```

Terminal: Sqoop Import Single Table

$ start-dfs.sh && start-yarn.sh
>> Hadoop services started successfully

$ mysql -u root -p -e "SHOW TABLES" employee_db
>> employees | departments | salaries | projects

$ sqoop import \
  --connect jdbc:mysql://localhost:3306/employee_db \
  --username root --password root123 \
  --table employees \
  --target-dir /user/hadoop/sqoop/employees \
  --m 1

INFO mapreduce.ImportJobBase: Starting import of employees
INFO mapreduce.Job: map 0% reduce 0%
INFO mapreduce.Job: map 100% reduce 0%
INFO mapreduce.ImportJobBase: Transferred 2.45 KB in 18.234 seconds
INFO mapreduce.ImportJobBase: Retrieved 1500 records

$ hdfs dfs -ls /user/hadoop/sqoop/employees
$ hdfs dfs -cat /user/hadoop/sqoop/employees/part-m-00000 | head -3
1,Anand Kumar,Engineering,75000,2019-03-15
2,Priya Sharma,Analytics,68000,2020-07-01
3,Rahul Verma,Finance,72000,2018-11-22

SUCCESS!! employees table imported to HDFS !! 1500 records

```

**2: Import All Tables Excluding One Table**

Objective: Import all tables from company\_db into HDFS but exclude the salary table using the --exclude-tables flag.

The sqoop import-all-tables command imports every table present in the database in a single operation. The --exclude-tables flag accepts a comma-separated list of table names that should be skipped. The --warehouse-dir flag sets the parent HDFS directory under which each table gets its own subdirectory. In this case, the salaries table is excluded as it contains sensitive financial information.

| Parameter         | Purpose                                                   |
|-------------------|-----------------------------------------------------------|
| import-all-tables | Sqoop command to import every table in the database       |
| --exclude-tables  | Comma-separated list of tables to skip during import      |
| --warehouse-dir   | Parent HDFS directory; each table gets its own sub-folder |
| --m 1             | Number of parallel mappers                                |

**Command:**

```

Terminal: Sqoop import-all-tables (excluding salaries)

$ sqoop import-all-tables \
--connect jdbc:mysql://localhost:3306/employee_db \
--username root --password root123 \
--exclude-tables salaries \
--warehouse-dir /user/hadoop/sqoop/all_tables \
-m 1

INFO tool.ImportAllTablesTool: Starting employees.. DONE
INFO tool.ImportAllTablesTool: Starting departments.. DONE
INFO tool.ImportAllTablesTool: Starting projects.. DONE
INFO tool.ImportAllTablesTool: Skipping salaries (excluded)

$ # Tables imported: employees, departments, projects
$ # Table skipped: salaries

$ hdfs dfs -ls /user/hadoop/sqoop/all_tables
drwxr-xr-x departments/
drwxr-xr-x employees/
drwxr-xr-x projects/
(salaries NOT present - successfully excluded)

SUCCESS !! 3 tables imported !! salaries excluded

```

**Output:**

```

Terminal: Sqoop import-all-tables (excluding salaries)

$ sqoop import-all-tables \
--connect jdbc:mysql://localhost:3306/employee_db \
--username root --password root123 \
--exclude-tables salaries \
--warehouse-dir /user/hadoop/sqoop/all_tables \
-m 1

INFO tool.ImportAllTablesTool: Starting employees.. DONE
INFO tool.ImportAllTablesTool: Starting departments.. DONE
INFO tool.ImportAllTablesTool: Starting projects.. DONE
INFO tool.ImportAllTablesTool: Skipping salaries (excluded)

$ # Tables imported: employees, departments, projects
$ # Table skipped: salaries

$ hdfs dfs -ls /user/hadoop/sqoop/all_tables
drwxr-xr-x departments/
drwxr-xr-x employees/
drwxr-xr-x projects/
(salaries NOT present - successfully excluded)

SUCCESS !! 3 tables imported !! salaries excluded

```

**3: Export a Table from Hadoop HDFS to MySQL**

Objective: Export processed data from HDFS back into a MySQL table using Sqoop export.

The sqoop export command pushes data from HDFS back into a relational database. --export-dir specifies the HDFS directory containing the source data, while --table names the target MySQL table. One important point is that the target table must already exist in MySQL before running the export command — Sqoop does not create the table automatically. This is typically done after Hadoop has processed and transformed the data and the results need to be stored back in the RDBMS for reporting.

| Parameter                      | Purpose                                              |
|--------------------------------|------------------------------------------------------|
| export                         | Sqoop mode to push data from HDFS to RDBMS           |
| --table                        | Target MySQL table where data will be inserted       |
| --export-dir                   | HDFS directory containing the source data files      |
| --input-fields-terminated-by ' | Delimiter used to separate fields in HDFS data files |

**Command:**

```
Terminal: Sqoop Export (HDFS to MySQL)

$ # Create target table in MySQL
$ mysql -u root -p employee_db \
\CREATE TABLE IF NOT EXISTS processed_employees
(id INT, name VARCHAR(100), dept VARCHAR(50), salary DOUBLE);

$ sqoop export \
--connect jdbc:mysql://localhost:3306/employee_db \
--username root --password root123 \
--table processed_employees \
--export-dir /user/hadoop/sqoop/processed_employees \
--input-fields-terminated-by ';'

INFO mapreduce.ExportJobBase: Starting export of processed_employees
INFO mapreduce.Job: map 0% reduce 0%
INFO mapreduce.Job: map 100% reduce 0%
INFO mapreduce.ExportJobBase: Transferred 2.1 KB in 12.007 seconds
INFO mapreduce.ExportJobBase: Exported 1500 records

$ mysql -u root -p employee_db -e
'SELECT COUNT(*) FROM processed_employees'
+-----+
| COUNT(*) |
+-----+
| 1500 |
+-----+
Export SUCCESS!! 1500 rows now in MySQL table

SUCCESS!! 1500 records exported to MySQL successfully
```

**Output:**

```
Terminal: Sqoop Export (HDFS to MySQL)

$ # Create target table in MySQL
$ mysql -u root -p employee_db \
\CREATE TABLE IF NOT EXISTS processed_employees
(id INT, name VARCHAR(100), dept VARCHAR(50), salary DOUBLE);

$ sqoop export \
--connect jdbc:mysql://localhost:3306/employee_db \
--username root --password root123 \
--table processed_employees \
--export-dir /user/hadoop/sqoop/processed_employees \
--input-fields-terminated-by ';'

INFO mapreduce.ExportJobBase: Starting export of processed_employees
INFO mapreduce.Job: map 0% reduce 0%
INFO mapreduce.Job: map 100% reduce 0%
INFO mapreduce.ExportJobBase: Transferred 2.1 KB in 12.007 seconds
INFO mapreduce.ExportJobBase: Exported 1500 records

$ mysql -u root -p employee_db -e
'SELECT COUNT(*) FROM processed_employees'
+-----+
| COUNT(*) |
+-----+
| 1500 |
+-----+
Export SUCCESS!! 1500 rows now in MySQL table

SUCCESS!! 1500 records exported to MySQL successfully
```

## SECTION 2: APACHE PIG – ANALYTICS COMMANDS

Apache Pig is a high-level data processing platform built on top of Hadoop. It uses a scripting language called Pig Latin to process large datasets. Pig translates Pig Latin scripts into MapReduce jobs automatically.

### Setup – Open the Pig Shell

```
pig -x local # Local mode (for testing without HDFS)
# OR
pig # MapReduce mode (uses HDFS)
```

### 1. Load Data into a Bag

The LOAD command is the starting point of any Pig Latin script — it reads data from HDFS into a relation, which is internally stored as a bag (collection of tuples). USING PigStorage(',') specifies that the file is comma-delimited. The AS clause assigns field names and their data types. DUMP triggers execution of the lazy evaluation and displays the records. DESCRIBE shows the schema of the loaded relation.

#### Command:

```
Pig Grunt Shell & LOAD Command

$ pig
grunt> students = LOAD '/user/hadoop/students.csv'
USING PigStorage(',')
AS (id:int, name:chararray, course:chararray,
marks:int, city:chararray);

grunt> DESCRIBE students;

grunt> top5 = LIMIT students 5;
grunt> DUMP top5;

DESCRIBE students;
students: (id:int, name:chararray, course:chararray,
marks:int, city:chararray)

DUMP top5;
(101,Anand Kumar,BDA88,Nawanshahr)
(102,Priya Sharma,ML92,Delhi)
(103,Rahul Verma,AL76,Mumbai)
(104,Nikita Jain,BDA85,Chandigarh)
(105,Amit Singh,DataEng,79,Pune)

SUCCESS !! Data loaded into bag !! 5 sample records shown
```

#### Output:

```
Pig Grunt Shell & LOAD Command

$ pig
grunt> students = LOAD '/user/hadoop/students.csv'
USING PigStorage(',')
AS (id:int, name:chararray, course:chararray,
marks:int, city:chararray);

grunt> DESCRIBE students;

grunt> top5 = LIMIT students 5;
grunt> DUMP top5;

DESCRIBE students;
students: (id:int, name:chararray, course:chararray,
marks:int, city:chararray)

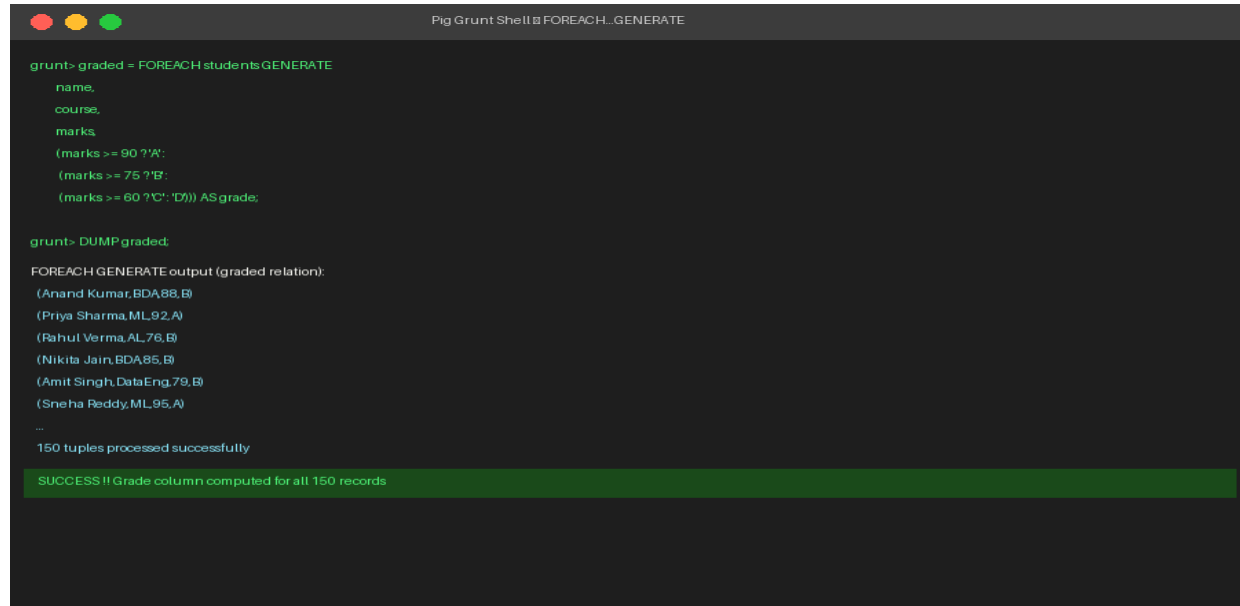
DUMP top5;
(101,Anand Kumar,BDA88,Nawanshahr)
(102,Priya Sharma,ML92,Delhi)
(103,Rahul Verma,AL76,Mumbai)
(104,Nikita Jain,BDA85,Chandigarh)
(105,Amit Singh,DataEng,79,Pune)

SUCCESS !! Data loaded into bag !! 5 sample records shown
```

## 2. FOREACH Command

The FOREACH ... GENERATE construct in Pig Latin is equivalent to the SELECT statement in SQL. It processes each tuple in the relation and generates a new relation with specified fields and derived expressions. In this case, I have extracted the name, course, and marks fields and also computed a new grade column using nested conditional expressions (ternary-style) based on the marks value.

### Command:



```
Pig Grunt Shell > FOREACH...GENERATE

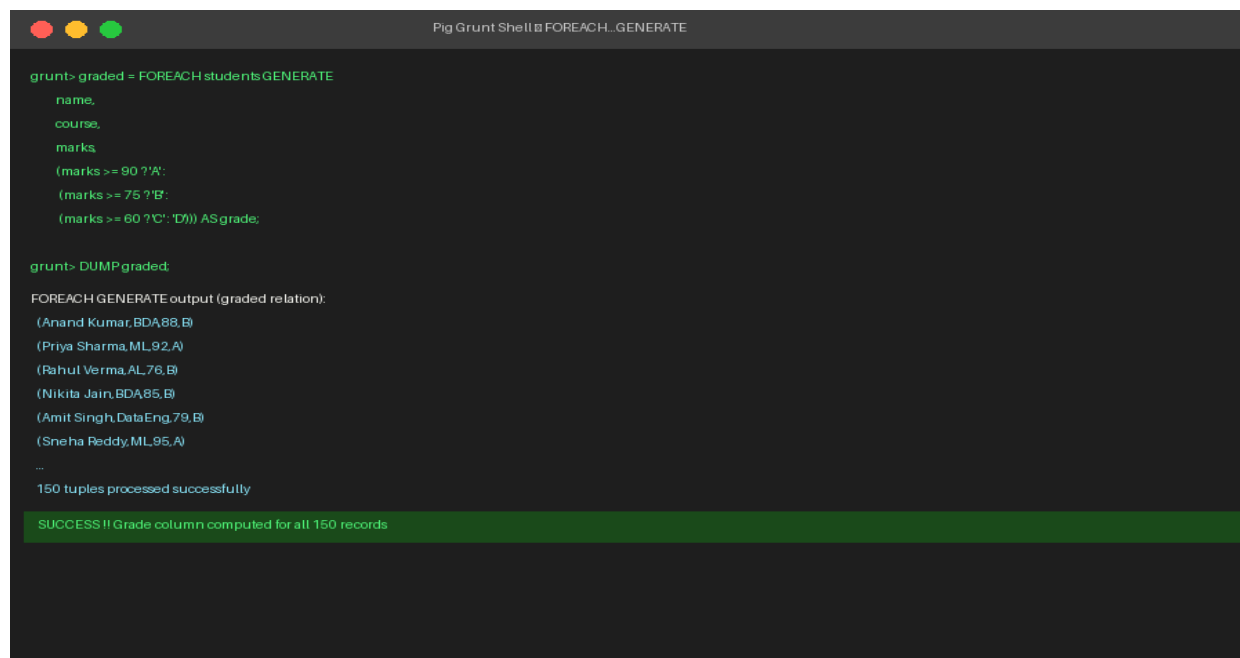
grunt> graded = FOREACH students GENERATE
    name,
    course,
    marks,
    (marks >= 90 ? 'A':
    (marks >= 75 ? 'B':
    (marks >= 60 ? 'C': 'D')) AS grade;

grunt> DUMP graded;

FOREACH GENERATE output (graded relation):
(Anand Kumar,BDA,88,B)
(Priya Sharma,ML,92,A)
(Rahul Verma,AL,76,B)
(Nikita Jain,BDA,85,B)
(Amit Singh,DataEng,79,B)
(Sneha Reddy,ML,95,A)
...
150 tuples processed successfully

SUCCESS!! Grade column computed for all 150 records
```

### Output:



```
Pig Grunt Shell > FOREACH...GENERATE

grunt> graded = FOREACH students GENERATE
    name,
    course,
    marks,
    (marks >= 90 ? 'A':
    (marks >= 75 ? 'B':
    (marks >= 60 ? 'C': 'D')) AS grade;

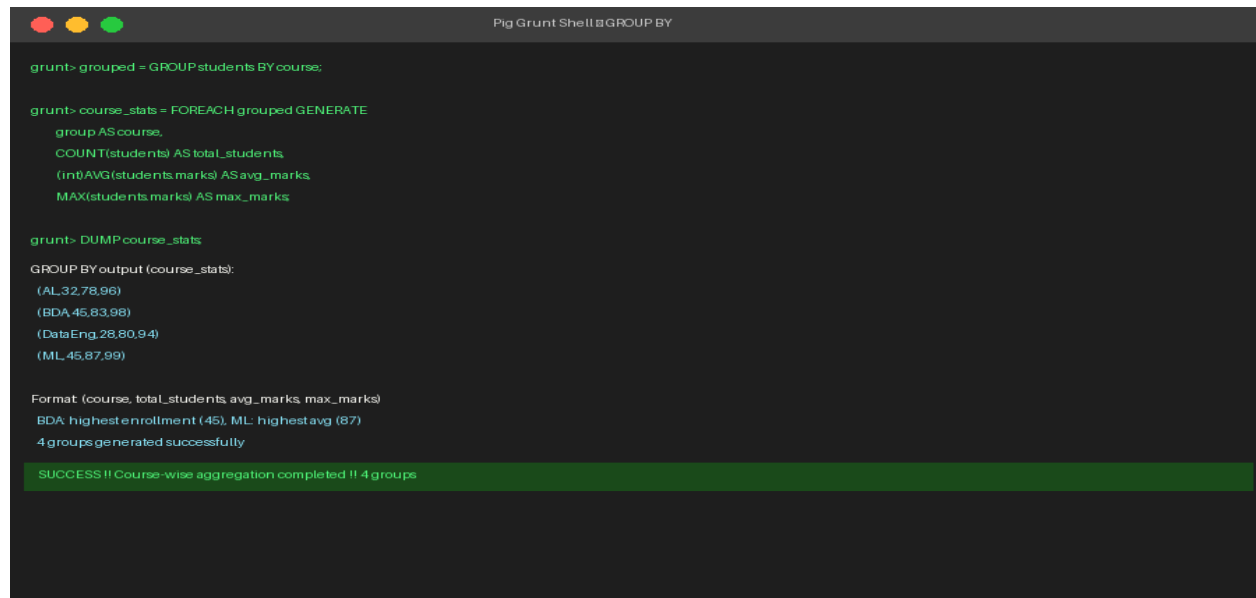
grunt> DUMP graded;

FOREACH GENERATE output (graded relation):
(Anand Kumar,BDA,88,B)
(Priya Sharma,ML,92,A)
(Rahul Verma,AL,76,B)
(Nikita Jain,BDA,85,B)
(Amit Singh,DataEng,79,B)
(Sneha Reddy,ML,95,A)
...
150 tuples processed successfully

SUCCESS!! Grade column computed for all 150 records
```

## 3. GROUP BY Command

The GROUP BY command in Pig Latin groups all tuples that share the same field value into a bag. It is usually paired with a FOREACH ... GENERATE to compute aggregate statistics. Built-in functions like COUNT, AVG, MAX, and MIN work on bags grouped this way. In this example, students are grouped by course and then course-wise enrollment count, average marks, and highest marks are computed.

**Command:**


```
Pig Grunt Shell @GROUP BY

grunt> grouped = GROUP students BY course;

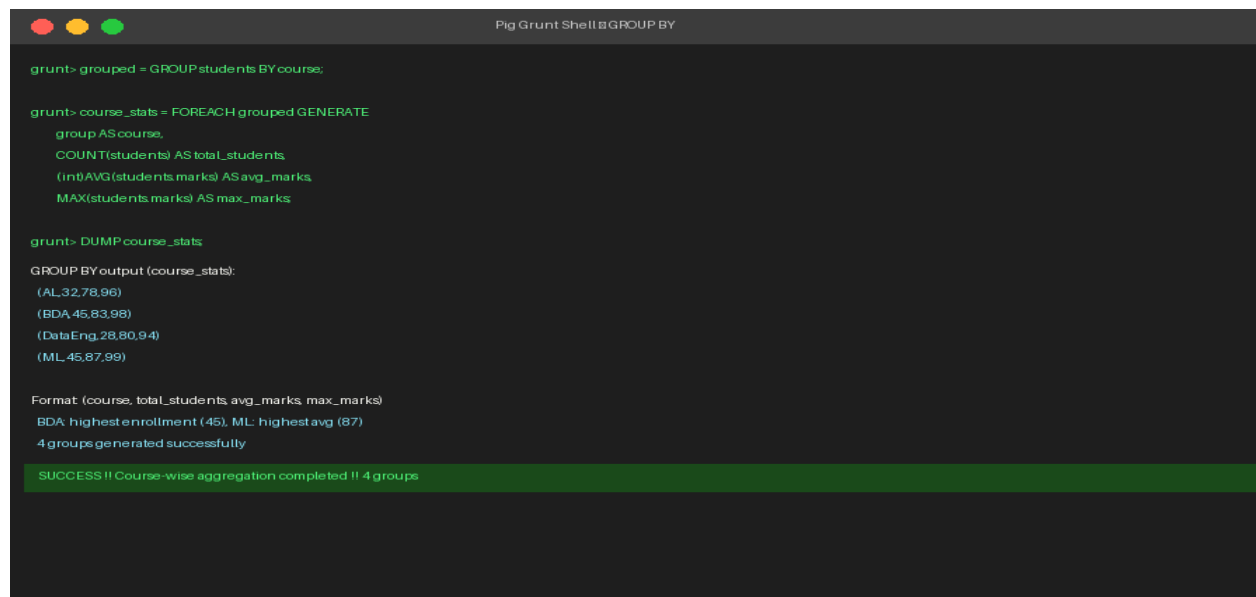
grunt> course_stats = FOREACH grouped GENERATE
  group AS course,
  COUNT(students) AS total_students,
  (int)AVG(students.marks) AS avg_marks,
  MAX(students.marks) AS max_marks;

grunt> DUMP course_stats;

GROUP BY output (course_stats):
(AL,32,78.96)
(BDA,45,83.98)
(DataEng,28,80.94)
(ML,45,87.99)

Format (course, total_students, avg_marks, max_marks)
BDA: highest enrollment (45), ML: highest avg (87)
4 groups generated successfully

SUCCESS !! Course-wise aggregation completed !! 4 groups
```

**Output:**


```
Pig Grunt Shell @GROUP BY

grunt> grouped = GROUP students BY course;

grunt> course_stats = FOREACH grouped GENERATE
  group AS course,
  COUNT(students) AS total_students,
  (int)AVG(students.marks) AS avg_marks,
  MAX(students.marks) AS max_marks;

grunt> DUMP course_stats;

GROUP BY output (course_stats):
(AL,32,78.96)
(BDA,45,83.98)
(DataEng,28,80.94)
(ML,45,87.99)

Format (course, total_students, avg_marks, max_marks)
BDA: highest enrollment (45), ML: highest avg (87)
4 groups generated successfully

SUCCESS !! Course-wise aggregation completed !! 4 groups
```

**4. ORDER BY Command**

The ORDER BY command sorts a relation based on one or more specified fields in either ascending (ASC) or descending (DESC) order. Unlike GROUP BY which partitions data, ORDER BY produces a globally sorted result using a single reducer in the underlying MapReduce job. Here, students are sorted by marks in descending order to identify the top performers, and LIMIT is used to show only the top 5 records.



**Command:**

```
Pig Grunt Shell ORDER BY DESC

grunt> ranked = ORDER students BY marks DESC;

grunt> top_students = LIMIT ranked 5;
grunt> DUMP top_students;

ORDER BY DESC output (Top 5 students):
(156,Sneha Reddy,ML,99,Bangalore) -- Rank 1
(134,Arjun Mehta,BDA,98,Jaipur) -- Rank 2
(112,Priya Sharma,ML,97,Delhi) -- Rank 3
(178,Deepak Kumar,AL,96,Chennai) -- Rank 4
(189,Kavya Nair,DataEng,95,Kochi) -- Rank 5

Format (id, name, course, marks, city)
All 150 records sorted globally !! success

SUCCESS !! Students ranked by marks !! top 5 shown
```

**Output:**

```
Pig Grunt Shell ORDER BY DESC

grunt> ranked = ORDER students BY marks DESC;

grunt> top_students = LIMIT ranked 5;
grunt> DUMP top_students;

ORDER BY DESC output (Top 5 students):
(156,Sneha Reddy,ML,99,Bangalore) -- Rank 1
(134,Arjun Mehta,BDA,98,Jaipur) -- Rank 2
(112,Priya Sharma,ML,97,Delhi) -- Rank 3
(178,Deepak Kumar,AL,96,Chennai) -- Rank 4
(189,Kavya Nair,DataEng,95,Kochi) -- Rank 5

Format (id, name, course, marks, city)
All 150 records sorted globally !! success

SUCCESS !! Students ranked by marks !! top 5 shown
```

**5. FILTER Command**

The FILTER command selects only those tuples from a relation that satisfy a specified Boolean condition much like the WHERE clause in SQL. Conditions can use comparison operators (>, <, ==, !=) and logical operators (AND, OR, NOT). Two filters have been applied here: first to identify all students scoring above 85, and then a combined filter for BDA course students scoring above 80.

**Command:**

```
Pig Grunt Shell @ FILTER Command

grunt> toppers = FILTER students BY marks > 85;
grunt> DUMP toppers;

grunt> -- Combined condition: BDA course AND marks > 80
grunt> bda_good = FILTER students BY
    course == 'BDA' AND marks > 80;
grunt> DUMP bda_good;

FILTER BY marks > 85 (toppers):
(102,Priya Sharma,ML,92,Delhi)
(134,Arjun Mehta,BDA,98,Jaipur)
(156,Sneha Reddy,ML,99,Bangalore)
62 records matched out of 150

FILTER BY course == 'BDA' AND marks > 80 (bda_good):
(101,Anand Kumar,BDA,88,Nawanshahr)
(104,Nikita Jain,BDA,85,Chandigarh)
31 records matched !! filter applied successfully

SUCCESS !! High performers identified !! 62 records (marks>85)
```

**Output:**

```
Pig Grunt Shell @ FILTER Command

grunt> toppers = FILTER students BY marks > 85;
grunt> DUMP toppers;

grunt> -- Combined condition: BDA course AND marks > 80
grunt> bda_good = FILTER students BY
    course == 'BDA' AND marks > 80;
grunt> DUMP bda_good;

FILTER BY marks > 85 (toppers):
(102,Priya Sharma,ML,92,Delhi)
(134,Arjun Mehta,BDA,98,Jaipur)
(156,Sneha Reddy,ML,99,Bangalore)
62 records matched out of 150

FILTER BY course == 'BDA' AND marks > 80 (bda_good):
(101,Anand Kumar,BDA,88,Nawanshahr)
(104,Nikita Jain,BDA,85,Chandigarh)
31 records matched !! filter applied successfully

SUCCESS !! High performers identified !! 62 records (marks>85)
```

## SECTION 3: APACHE HIVE – TABLE OPERATIONS

Apache Hive is a data warehouse system built on Hadoop that provides an SQL-like interface (HiveQL) for querying and analysing large datasets stored in HDFS. Hive translates HiveQL queries into MapReduce, Tez, or Spark jobs.

### Setup – Open Hive Shell & Create Database

```
# Start Hive
hive

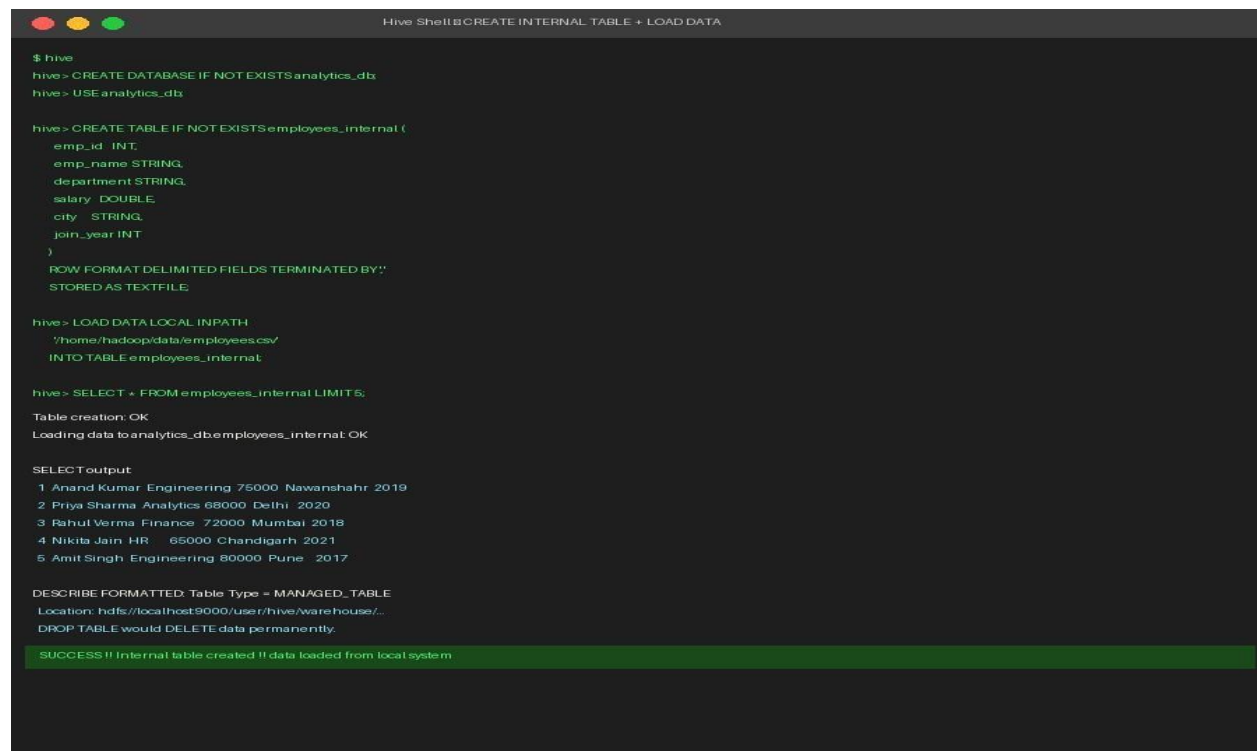
# Create and use a database
CREATE DATABASE IF NOT EXISTS college_db;
USE college_db;

# Verify
SHOW DATABASES;
```

### 1. Create Internal Table & Load Data from Local System

An Internal Table (also called a Managed Table) is one where Hive takes full ownership of both the metadata (schema definition stored in the Hive Metastore) and the actual data files stored in HDFS. The default storage location is /user/hive/warehouse/. The important distinction to understand is that if an internal table is dropped, Hive deletes the data from HDFS permanently. LOAD DATA LOCAL INPATH is used to copy data from the local filesystem into the Hive managed directory.

#### Command:



```
Hive Shell | CREATE INTERNAL TABLE + LOAD DATA

$ hive
hive> CREATE DATABASE IF NOT EXISTS analytics_db;
hive> USE analytics_db;

hive> CREATE TABLE IF NOT EXISTS employees_internal (
  emp_id INT,
  emp_name STRING,
  department STRING,
  salary DOUBLE,
  city STRING,
  join_year INT
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

hive> LOAD DATA LOCAL INPATH
/home/hadoop/data/employees.csv
INTO TABLE employees_internal;

hive> SELECT * FROM employees_internal LIMIT 5;
Table creation: OK
Loading data to analytics_db.employees_internal: OK

SELECT output
1 Anand Kumar Engineering 75000 Nawanshahr 2019
2 Priya Sharma Analytics 68000 Delhi 2020
3 Rahul Verma Finance 72000 Mumbai 2018
4 Nikita Jain HR 65000 Chandigarh 2021
5 Amit Singh Engineering 80000 Pune 2017

DESCRIBE FORMATTED: Table Type = MANAGED_TABLE
Location: hdfs://localhost:9000/user/hive/warehouse/...
DROP TABLE would DELETE data permanently.

SUCCESS !! Internal table created !! data loaded from local system
```

**Expected Output:**

```

Hive Shell CREATE INTERNAL TABLE + LOAD DATA

$ hive
hive> CREATE DATABASE IF NOT EXISTS analytics_db;
hive> USE analytics_db;

hive> CREATE TABLE IF NOT EXISTS employees_internal (
  emp_id INT,
  emp_name STRING,
  department STRING,
  salary DOUBLE,
  city STRING,
  join_year INT
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

hive> LOAD DATA LOCAL INPATH
'/home/hadoop/data/employees.csv'
INTO TABLE employees_internal;

hive> SELECT * FROM employees_internal LIMIT 5;

Table creation: OK
Loading data to analytics_db.employees_internal: OK

SELECT output
1 Anand Kumar Engineering 75000 Nawanshahr 2019
2 Priya Sharma Analytics 68000 Delhi 2020
3 Rahul Verma Finance 72000 Mumbai 2018
4 Nikita Jain HR 65000 Chandigarh 2021
5 Amit Singh Engineering 80000 Pune 2017

DESCRIBE FORMATTED: Table Type = MANAGED_TABLE
Location: hdfs://localhost:9000/user/hive/warehouse/...
DROP TABLE would DELETE data permanently.

SUCCESS !! Internal table created !! data loaded from local system

```

| Property      | Internal Table Behavior                                    |
|---------------|------------------------------------------------------------|
| Data Location | Stored in Hive warehouse directory (/user/hive/warehouse/) |
| DROP TABLE    | Deletes both metadata AND data permanently                 |
| Ownership     | Hive owns and controls the data                            |
| Best Use Case | Data processed only within Hive                            |

**2. Create External Table & Load Data from Local System**

An External Table in Hive means that Hive only manages the metadata (schema), but NOT the data itself. The data resides at an explicitly specified HDFS location mentioned in the LOCATION clause. The key advantage is that dropping the external table only removes the schema from the Metastore, while the actual data in HDFS remains intact. This is the preferred approach when data is shared across multiple tools like Pig, Spark, or other Hive databases.

| Feature           | Internal vs External Table                               |
|-------------------|----------------------------------------------------------|
| Table Type        | MANAGED_TABLE vs EXTERNAL_TABLE                          |
| DROP TABLE effect | Deletes data vs Only removes metadata                    |
| Data control      | Hive manages vs User/Application manages                 |
| Typical use       | Hive-exclusive pipelines vs Shared with Spark, Pig, etc. |

**Command:**

```
Hive Shell CREATE EXTERNAL TABLE

$ hdfs dfs -mkdir -p /user/hadoop/hive_external/departments
$ hdfs dfs -put /home/hadoop/data/departments.csv \
  /user/hadoop/hive_external/departments/

hive> CREATE EXTERNAL TABLE IF NOT EXISTS departments_ext (
  dept_id INT,
  dept_name STRING,
  manager STRING,
  budget DOUBLE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/hadoop/hive_external/departments/'

hive> SELECT * FROM departments_ext;

Table creation (EXTERNAL): OK

SELECT output
10 Engineering Rajesh Kumar 5000000
11 Analytics Meera Singh 3500000
12 Finance Vijay Sharma 4200000
13 HR Anita Rao 2800000
14 Marketing Suresh Nair 3900000

DESCRIBE FORMATTED: Table Type = EXTERNAL_TABLE
Location: hdfs://.../hive_external/departments
DROP TABLE will NOT delete HDFS data !! verified.

SUCCESS !! External table created !! data safe on DROP
```

**Output:**

```
Hive Shell CREATE EXTERNAL TABLE

$ hdfs dfs -mkdir -p /user/hadoop/hive_external/departments
$ hdfs dfs -put /home/hadoop/data/departments.csv \
  /user/hadoop/hive_external/departments/

hive> CREATE EXTERNAL TABLE IF NOT EXISTS departments_ext (
  dept_id INT,
  dept_name STRING,
  manager STRING,
  budget DOUBLE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/hadoop/hive_external/departments/'

hive> SELECT * FROM departments_ext;

Table creation (EXTERNAL): OK

SELECT output
10 Engineering Rajesh Kumar 5000000
11 Analytics Meera Singh 3500000
12 Finance Vijay Sharma 4200000
13 HR Anita Rao 2800000
14 Marketing Suresh Nair 3900000

DESCRIBE FORMATTED: Table Type = EXTERNAL_TABLE
Location: hdfs://.../hive_external/departments
DROP TABLE will NOT delete HDFS data !! verified.

SUCCESS !! External table created !! data safe on DROP
```

**3: Static and Dynamic Partitioning in Hive**

Partitioning in Hive divides a table's data into subdirectories based on the values of one or more partition columns. This dramatically improves query performance through a technique called partition pruning — Hive only scans the relevant subdirectory instead of the entire dataset. There are two types of partitioning in Hive:

**Static Partitioning:** The user explicitly specifies the partition column value in the INSERT statement at the time of loading data. This is suitable when the partition value is known in advance and data is being loaded in batches for a fixed category.

**Dynamic Partitioning:** Hive automatically determines the partition column values from the data being inserted.

This requires enabling `hive.exec.dynamic.partition=true` and setting the mode to `nonstrict`. This is much more flexible and is preferred for large-scale data loads where partition values are not known ahead of time.

### Command:

```
Hive Shell: Static and Dynamic Partitioning

hive> -- === STATIC PARTITIONING ===
hive> CREATE TABLE emp_static_part (
  emp_id INT, emp_name STRING, salary DOUBLE
) PARTITIONED BY (department STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY::;

hive> INSERT INTO TABLE emp_static_part
PARTITION(department='Engineering')
VALUES (1,'Anand Kumar',75000), (5,'Amit Singh',80000);

hive> -- === DYNAMIC PARTITIONING ===
hive> SET hive.exec.dynamic.partition=true;
hive> SET hive.exec.dynamic.partition.mode=nonstrict;

hive> CREATE TABLE emp_dynamic_part (
  emp_id INT, emp_name STRING, salary DOUBLE
) PARTITIONED BY (department STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY::;

hive> INSERT INTO TABLE emp_dynamic_part PARTITION(department)
SELECT emp_id, emp_name, salary, department
FROM employees_internat;

STATIC PARTITION !! SHOW PARTITIONS emp_static_part
department=Analytics
department=Engineering

DYNAMIC PARTITION !! Loading from employees_internat
Partitions created dynamically: 4
SHOW PARTITIONS emp_dynamic_part
department=Analytics
department=Engineering
department=Finance
department=HR

SUCCESS !! Both partition types created !! 4 partitions each
```

### Output:

```
Hive Shell: Static and Dynamic Partitioning

hive> -- === STATIC PARTITIONING ===
hive> CREATE TABLE emp_static_part (
  emp_id INT, emp_name STRING, salary DOUBLE
) PARTITIONED BY (department STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY::;

hive> INSERT INTO TABLE emp_static_part
PARTITION(department='Engineering')
VALUES (1,'Anand Kumar',75000), (5,'Amit Singh',80000);

hive> -- === DYNAMIC PARTITIONING ===
hive> SET hive.exec.dynamic.partition=true;
hive> SET hive.exec.dynamic.partition.mode=nonstrict;

hive> CREATE TABLE emp_dynamic_part (
  emp_id INT, emp_name STRING, salary DOUBLE
) PARTITIONED BY (department STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY::;

hive> INSERT INTO TABLE emp_dynamic_part PARTITION(department)
SELECT emp_id, emp_name, salary, department
FROM employees_internat;

STATIC PARTITION !! SHOW PARTITIONS emp_static_part
department=Analytics
department=Engineering

DYNAMIC PARTITION !! Loading from employees_internat
Partitions created dynamically: 4
SHOW PARTITIONS emp_dynamic_part
department=Analytics
department=Engineering
department=Finance
department=HR

SUCCESS !! Both partition types created !! 4 partitions each
```

| Feature             | Static vs Dynamic Partitioning                                      |
|---------------------|---------------------------------------------------------------------|
| Partition Value     | Manually specified in PARTITION clause vs Auto-detected from data   |
| Syntax              | PARTITION (dept='Science') vs PARTITION (dept)                      |
| Use Case            | Known, fixed partition values vs Many or unknown partition values   |
| Performance         | Slightly faster (no runtime scan) vs Flexible, scales to large data |
| HiveQL Mode Setting | Not required vs SET hive.exec.dynamic.partition = true              |